

MODULE 1.1: Java refresher and Execution Basics

Topics

- Java program structure and compilation flow
- main method execution lifecycle
- variables, primitive types, and type conversion
- operators (logical, relational, arithmetic)
- control flow: if-else, switch, loops
- method declaration, parameters, return types

Practice Session

Build a **Student Grading System**:

- Accept multiple student marks
- Compute grades using conditions
- Modularize logic into methods
- Ensure clean input-output separation

Answer:

```
import java.util.Scanner;

public class StudentGradeSystem {

    // Method to accept student marks
    public static int[] getMarks(int n, Scanner sc) {
        int[] marks = new int[n];

        for (int i = 0; i < n; i++) {
            System.out.print("Enter marks for Student " + (i + 1) + ": ");
            marks[i] = sc.nextInt();
        }

        return marks;
    }
}
```

```

// Method to calculate grade
public static String calculateGrade(int marks) {

    if (marks >= 90)
        return "A+";
    else if (marks >= 80)
        return "A";
    else if (marks >= 70)
        return "B";
    else if (marks >= 60)
        return "C";
    else if (marks >= 50)
        return "D";
    else
        return "Fail";
}

// Method to display results
public static void displayResult(int[] marks) {

    System.out.println("\nStudent Results");
    System.out.println("-----");
    System.out.println("Marks\tGrade");

    for (int mark : marks) {
        System.out.println(mark + "\t" + calculateGrade(mark));
    }
}

public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    System.out.print("Enter number of students: ");
    int n = sc.nextInt();

    int[] marks = getMarks(n, sc);

    displayResult(marks);

    sc.close();
}
}

```

MODULE 1.2: Object-Oriented Programming Refresher

Topics

- Class and object lifecycle
- Encapsulation and access modifiers
- Constructors (default, parameterized, overloaded)
- Basic inheritance and method overriding
- this and super usage

Practice Session

Design an **Employee Management System**:

- Employee class with attributes
- Department mapping
- Salary calculation logic
- Proper encapsulation and object separation

Answer:

```
public class Department {  
  
    private int departmentId;  
    private String departmentName;  
  
    public Department(int departmentId, String departmentName) {  
        this.departmentId = departmentId;  
        this.departmentName = departmentName;  
    }  
  
    public int getDepartmentId() {  
        return departmentId;  
    }  
  
    public String getDepartmentName() {  
        return departmentName;  
    }  
}  
  
public class Employee {
```

```

private int empId;
private String empName;
private double basicSalary;
private Department department;

public Employee(int empId, String empName,
               double basicSalary,
               Department department) {

    this.empId = empId;
    this.empName = empName;
    this.basicSalary = basicSalary;
    this.department = department;
}

public double calculateSalary() {

    double hra = basicSalary * 0.20;
    double da = basicSalary * 0.10;

    return basicSalary + hra + da;
}

public void displayEmployee() {

    System.out.println("Employee Id : " + empId);
    System.out.println("Employee Name : " + empName);
    System.out.println("Department : "
        + department.getDepartmentName());

    System.out.println("Total Salary : "
        + calculateSalary());

    System.out.println("-----");
}
}

public class EmployeeManagementSystem {

    public static void main(String[] args) {

        Department it =
            new Department(101,"Information Technology");

```

```
    Department hr =  
        new Department(102,"Human Resources");  
  
    Employee e1 =  
        new Employee(1,"Rajani",50000,it);  
  
    Employee e2 =  
        new Employee(2,"Rahul",45000,hr);  
  
    e1.displayEmployee();  
    e2.displayEmployee();  
}  
}
```

MODULE 1.3: Memory Basics for Java Applications

Topics

- Stack vs heap memory model
- Object creation and reference behavior
- Garbage object identification (basic)
- Memory lifecycle in execution flow
- Why memory matters in backend systems

Practice Session

Trace execution of a Java program:

- Identify stack vs heap allocation
- Track object references
- Explain lifecycle of objects step-by-step

1. Odd or Even Sum

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int sum = 0;

        for (int i = 0; i < n; i++) {
            sum += sc.nextInt();
        }

        if (sum % 2 == 0)
            System.out.println("Even");
        else
            System.out.println("Odd");
    }
}
```

2. Prime Number Check

```
import java.util.*;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        if (n <= 1) {
            System.out.println("No");
            return;
        }

        boolean prime = true;

        for (int i = 2; i <= Math.sqrt(n); i++) {
```

```
        if (n % i == 0) {
            prime = false;
            break;
        }
    }

    System.out.println(prime ? "Yes" : "No");
}
}
```

3. Check Divisibility by 7

```
import java.util.*;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        if (n % 7 == 0)
            System.out.println("Yes");
        else
            System.out.println("No");
    }
}
```

4. Next Greater Power of 2

```
import java.util.*;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        int power = 1;
```

```
        while (power <= n) {  
            power *= 2;  
        }  
  
        System.out.println(power);  
    }  
}
```

5. Palindrome Check

```
import java.util.*;  
  
public class Main {  
    public static void main(String[] args) {  
  
        Scanner sc = new Scanner(System.in);  
  
        int n = sc.nextInt();  
        int temp = n;  
        int rev = 0;  
  
        while (n > 0) {  
            rev = rev * 10 + n % 10;  
            n /= 10;  
        }  
  
        if (temp == rev)  
            System.out.println("Yes");  
        else  
            System.out.println("No");  
    }  
}
```

6. Vowel Check in String

```
import java.util.*;  
  
public class Main {  
    public static void main(String[] args) {
```

```
Scanner sc = new Scanner(System.in);

String s = sc.nextLine().toLowerCase();

boolean found = false;

for (char c : s.toCharArray()) {
    if ("aeiou".indexOf(c) != -1) {
        found = true;
        break;
    }
}

System.out.println(found ? "Yes" : "No");
}
```

7. Number Range Check

```
import java.util.*;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        if (n >= 1 && n <= 100)
            System.out.println("Yes");
        else
            System.out.println("No");
    }
}
```

8. Round to Nearest Greater Integer

```
import java.util.*;

public class Main {
```

```
public static void main(String[] args) {  
  
    Scanner sc = new Scanner(System.in);  
  
    double n = sc.nextDouble();  
  
    System.out.println((int)Math.ceil(n));  
}  
}
```

9. String Middle Element Modification

```
import java.util.*;  
  
public class Main {  
    public static void main(String[] args) {  
  
        Scanner sc = new Scanner(System.in);  
  
        String s = sc.next();  
  
        int mid = s.length() / 2;  
  
        String ans = s.substring(0, mid) + "*" + s.substring(mid + 1);  
  
        System.out.println(ans);  
    }  
}
```

10. Check if Number is Composite

```
import java.util.*;  
  
public class Main {  
    public static void main(String[] args) {  
  
        Scanner sc = new Scanner(System.in);  
  
        int n = sc.nextInt();
```

```
boolean composite = false;

for (int i = 2; i <= Math.sqrt(n); i++) {
    if (n % i == 0) {
        composite = true;
        break;
    }
}

System.out.println(composite ? "Yes" : "No");
}
```

11. Print Factors of a Number

```
import java.util.*;

public class Main {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        for (int i = 1; i <= n; i++) {
            if (n % i == 0)
                System.out.print(i + " ");
        }
    }
}
```